Microsoft

The .NET Ecosystem For Sustainable Small Language Models with Foundry Local:

Tools, Techniques, and Tomorrow

By Charunthon Limseelo (Boat)

Microsoft Learn Student Ambassador
& Undergraduate Research Assistant at King Mongkut's University of Technology Thonburi

Session Speaker

Charunthon Limseelo (@boatchrnthn)

- Microsoft Learn Student Ambassador (Student Tech Lead) at Microsoft and Data Engineer Intern at Seven Peaks Software, Klong Toei, Bangkok, TH
- Speaker on Various International Stages such as PyCon TH, FOSSASIA, JavaScript Bangkok, BKK.JS, etc.
- Academic Research Assistant for Generative AI Development for KMUTT and Chulalongkorn University
- Published the first conference paper on 2025 22nd International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON), on the paper name: Thai Humor Generation on Small Language Model at Chulalongkorn University

Charunthon Limseelo (@boatchrnthn)



Session Outline

I. Introduction & The “Why”

II. .NET Tools & Techniques for Checking SLMs Sustainability

III. Conclusion & Q&A

Introduction & The “Why”

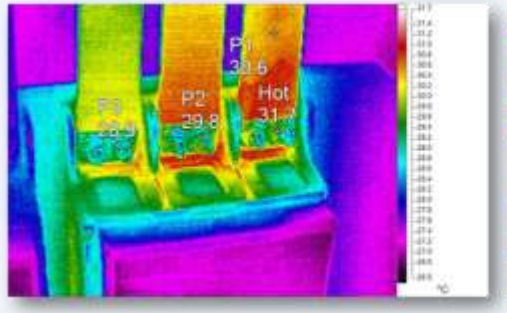


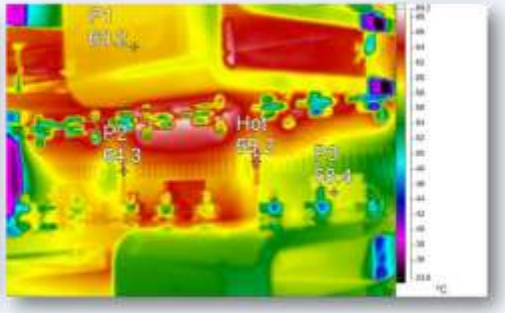
As an engineer...

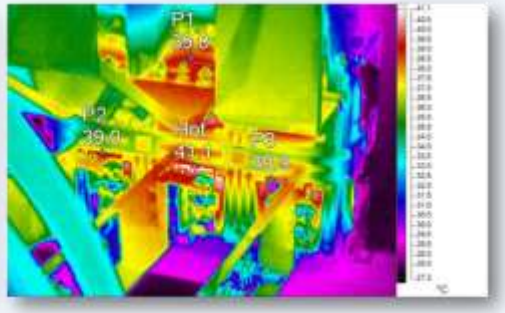
There is something
I would like to say









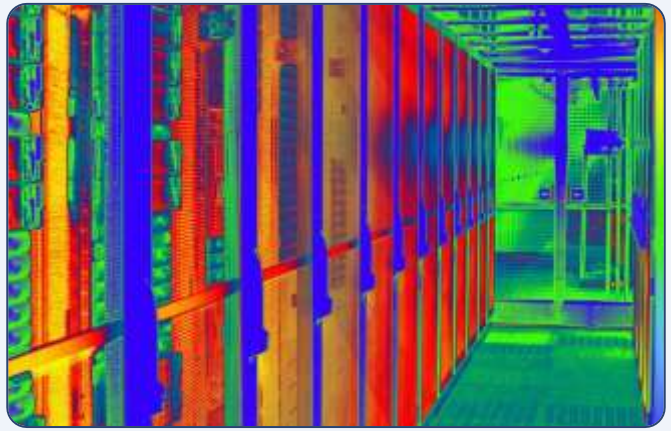


Majority Problems for Running LLMs

OpenAI Major Outage → Highly Computation → High Cost for Operation



OpenAI has experienced outages, disrupting services like ChatGPT due to factors like cloud provider failures or high demand.



GPT models need substantial computational resources, which can result in challenges with contextual understanding and responsiveness.



Running GPT models is expensive due to high hardware and electricity costs, making it difficult for smaller organizations to afford.

Proprietary and Open-Source Models

Proprietary

(adj.) used, made, or marketed by one having the exclusive legal right, privately owned and managed and run as a profit-making organization, Close-Source

Ex. OpenAI-GPT (Main Models), Gemini, Claude

Open Models

(adj.) having the source model freely available for possible modification and redistribution, along with publicly available for use by the community.



OpenAI GPT-oss, Hugging Face

Why Local and Small AI?

Throwback memory about Small Language Models



Privacy & security

– Local AI ensures that sensitive data remains on the device, minimizing exposure to external threats.

Latency

– Processing data locally reduces response time, making AI interactions faster and more efficient.

Cost efficient

– Running AI on-device can eliminate the need for expensive cloud services, lowering operational costs.

Low bandwidth

– Since data processing happens locally, minimal internet connectivity is required, reducing bandwidth usage.

Small Language Models Examples in Nowadays

Frontier Models with Multimodal Capabilities

Represent the bleeding edge of SLM capabilities. They are often developed by major AI labs and aim to push the boundaries of what's possible with a "smaller" footprint (though "small" can still mean billions of parameters) + Multimodal Ca.

Gemma 3

Microsoft

→ Phi-4 Multimodal Instruct

Normal Text Model or Reasoning-based

Encompasses the workhorse SLMs primarily focused on text-in, text-out generation and understanding. They are often the base models before extensive fine-tuning.

Microsoft

Meta

deepseek

Specialize-tuned

Result of taking a base SLM and extensively fine-tuning it on specific datasets for a narrow domain or task. This significantly improves their performance and relevance within that specialized area.



Localized Model

Addresses the crucial need for AI models that understand and cater to specific linguistic, cultural, and regional nuances. These models are essential for effective and equitable AI deployment globally.

SCB 10X

AI SINGAPORE

sea AI Lab

Model Formats in Nowadays

GGUF/GGML

- These formats are specifically designed for efficient execution of large language models on consumer-grade hardware, particularly CPUs, but also increasingly on GPUs, even with limited VRAM.

ONNX

- An open standard format for representing machine learning models. It's designed to be an interoperable format that allows models to be trained in one framework (e.g., PyTorch, TensorFlow) and deployed in another, often for inference optimization.

MLX

- A machine learning framework developed by Apple specifically for Apple Silicon (M-series chips). It's designed to provide an efficient and user-friendly way to train and deploy models directly on Apple's unified memory architecture. The "MLX format" typically refers to models saved using this framework's native serialization.

Safetensors/Torch

- A relatively new and increasingly popular format specifically designed for securely loading and sharing large deep learning models. It addresses security vulnerabilities present in traditional serialization formats like PyTorch's pickle-based state_dict files.

- Torch (PyTorch Checkpoints): Refers to the native serialization format for PyTorch models, typically saved as .pt or .pth files. These files contain the model's learned weights and biases (the state_dict) and sometimes the entire model graph.







The moment for Local AI is NOW

Not just for exploring, but for shipping real apps across platforms

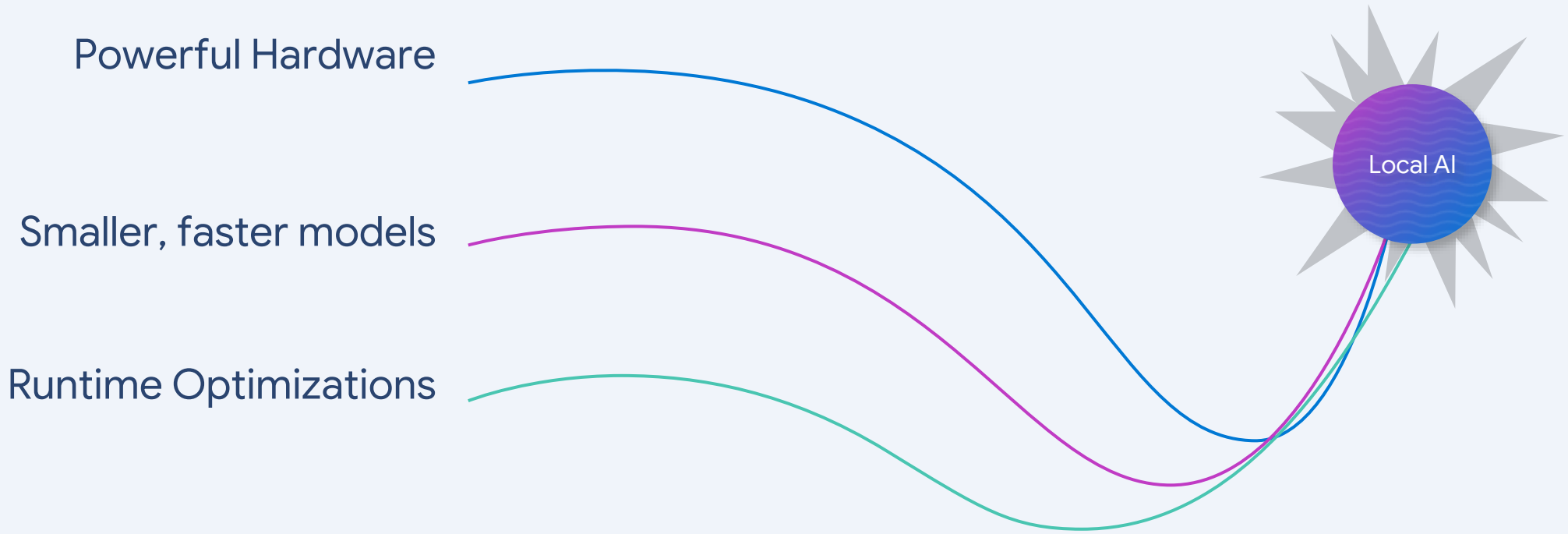


Singapore Airlines And Microsoft Partner To Enhance Customer Experience And Operational Efficiency With AI Solutions



Formula 1 driver, Pierre Gasly, on using custom Copilot on running within the enterprise and the BWT Alpine team for analyzing the race strategy.

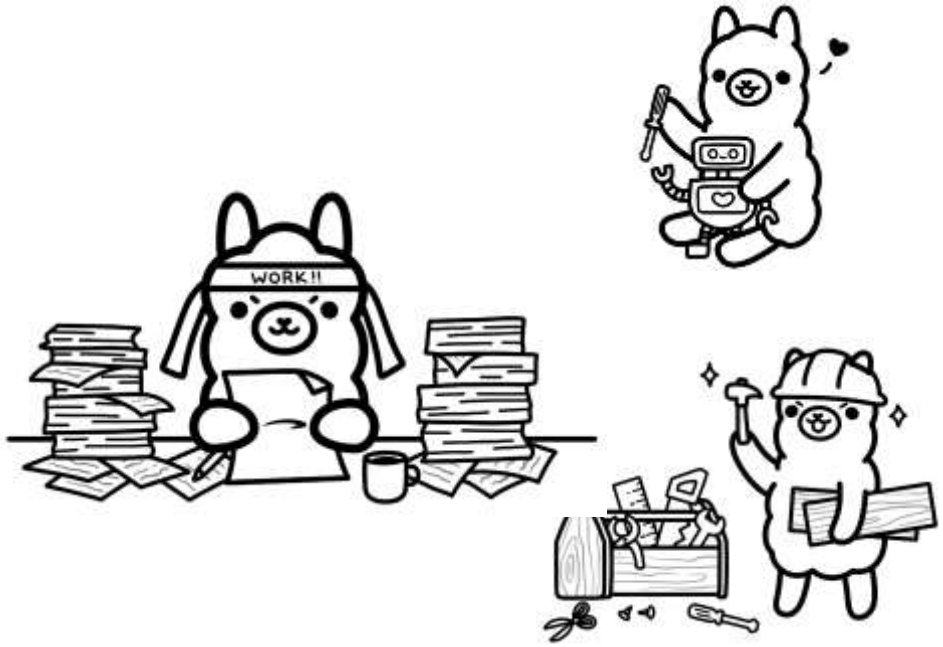
Local AI is now viable and delightful



Prerequisites for Running SLMs

- Operating System: Windows 10 (x64), Windows 11 (x64/ARM), Windows Server 2025, macOS 12 Monterey or later.
- Hardware: Minimum 8GB RAM, 3GB free disk space. Recommended 16GB RAM, 15GB free disk space.
- Network: Internet connection for initial model download (optional for offline use)
- Acceleration (optional): NVIDIA RTX GPU (2,000 series or newer), AMD GPU (6,000 series or newer), Intel iGPU, Qualcomm Snapdragon X Elite (8GB or more of memory), or Apple silicon.

Reimagining - Ollama



- Ollama is a lightweight, extensible framework for building and running language models on the local machine. It provides a simple API for creating, running, and managing models, as well as a library of pre-built models that can be easily used in a variety of applications.

```
ollama                                                    Update Warp

base ~/Code/ecosystem
ollama run zephyr
eval count:           317 token(s)
eval duration:        6.37296s
eval rate:            49.74 tokens/s
>>> what animals do not?
As I mentioned earlier, many animal species do not have clear and rigid social hierarchies among
adult individuals in their wild populations. Some examples include:

1. Polar bears: They are solitary animals that do not live in groups with a social hierarchy.

2. Bald eagles: While they form pairs during mating season, there is no dominant leader or hierarchy
within the pair or between pairs.

3. Sea otters: Although they form social groups, there is no hierarchy among them. The social
structure is more cooperative and based on sharing resources.

4. Tiger sharks: They are solitary predators that do not live in groups with a social hierarchy.

5. Orangutans: While they may form social groups led by a dominant female, there is generally less
rigid and hierarchical structure compared to some other primates such as chimpanzees, gorillas, and
bonobos.

These are just a few examples of animals that do not have clear social hierarchies among adult
individuals in their wild populations. However, it's important to note that the presence or absence
of social hierarchy can vary depending on factors such as species, environment, and context.

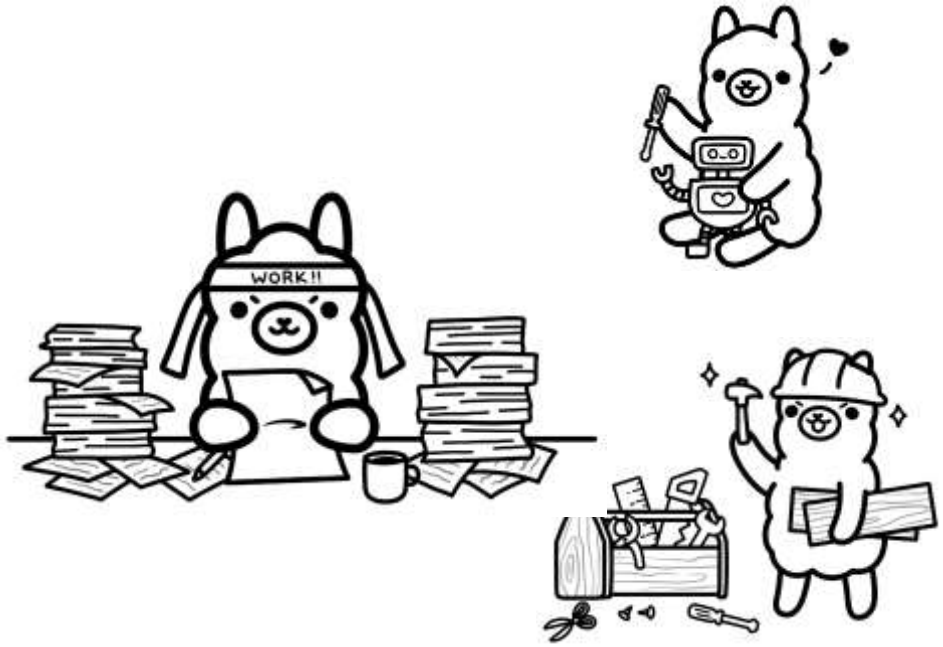
total duration:       8.592375916s
load duration:        1.528084666s
prompt eval count:    1169 token(s)
prompt eval duration: 2.050488s
prompt eval rate:     570.11 tokens/s
eval count:           255 token(s)
eval duration:        4.95492s
eval rate:            51.46 tokens/s
>>> Send a message (/? for help)
```



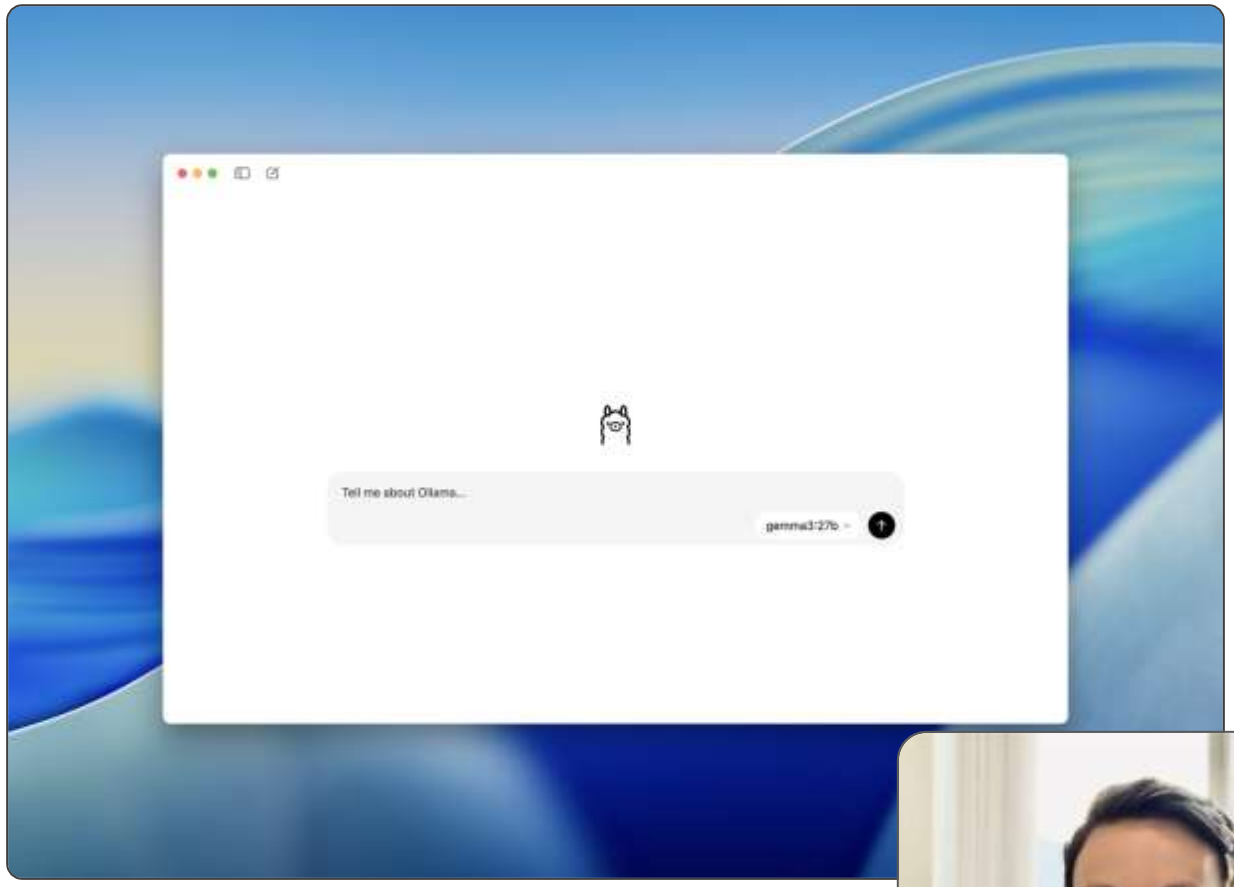
Jeffrey Morgan

Ollama Founder

Reimagining - Ollama



- Ollama is a lightweight, extensible framework for building and running language models on the local machine. It provides a simple API for creating, running, and managing models, as well as a library of pre-built models that can be easily used in a variety of applications.





Jeffrey Morgan

Ollama Founder

Using likely OpenAI API Compatibility

Which is help you to understand how the model communicate with your projects.

```
using Microsoft.Extensions.AI;
using OllamaSharp;

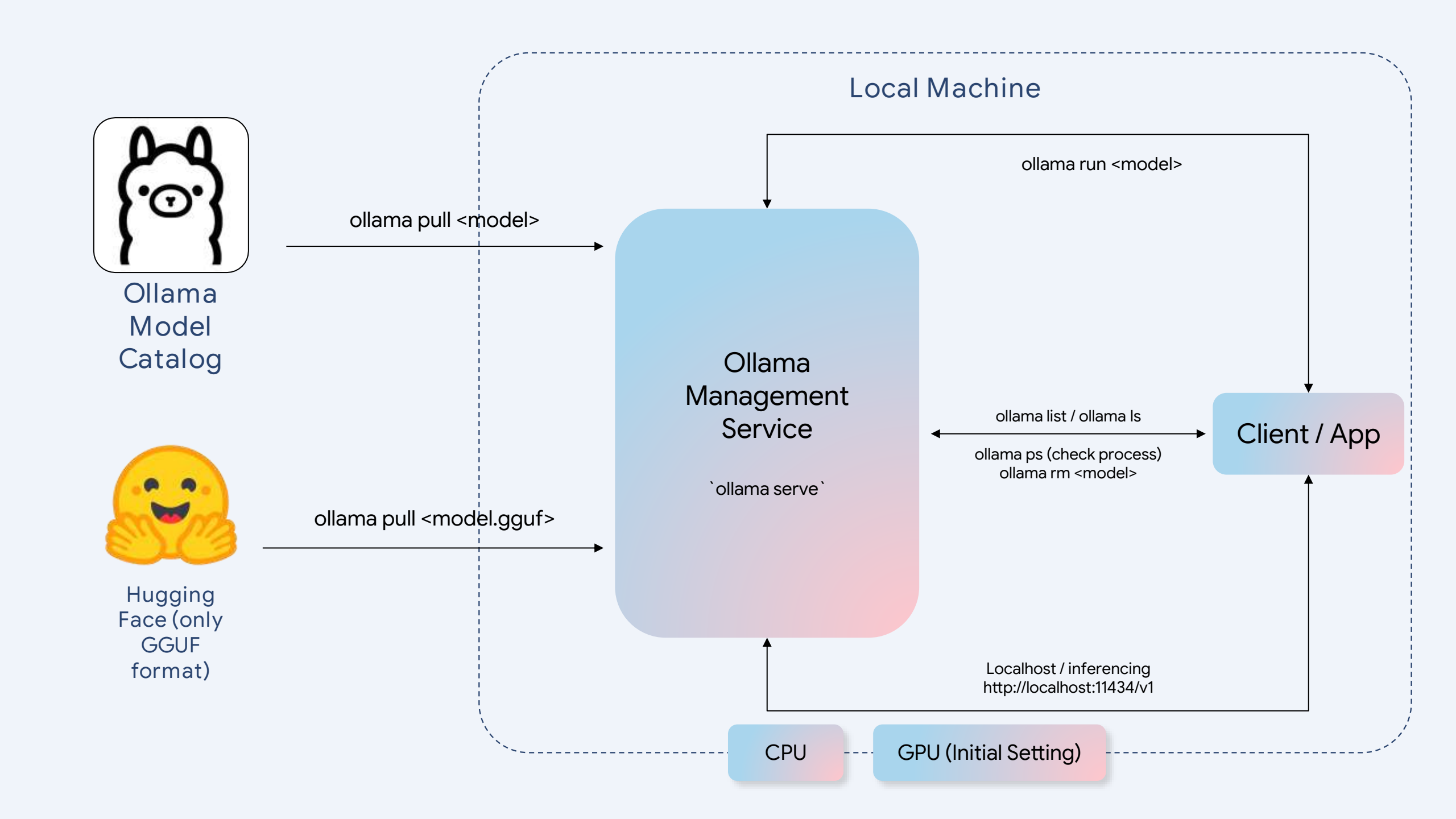
IChatClient chatClient =
    new OllamaApiClient(new Uri("http://localhost:11434/"), "phi3:mini");

// Start the conversation with context for the AI model
List<ChatMessage> chatHistory = new();

while (true)
{
    // Get user prompt and add to chat history
    Console.WriteLine("Your prompt:");
    var userPrompt = Console.ReadLine();
    chatHistory.Add(new ChatMessage(ChatRole.User, userPrompt));

    // Stream the AI response and add to chat history
    Console.WriteLine("AI Response:");
    var response = "";
    await foreach (ChatResponseUpdate item in
        chatClient.GetStreamingResponseAsync(chatHistory))
    {
        Console.Write(item.Text);
        response += item.Text;
    }
    chatHistory.Add(new ChatMessage(ChatRole.Assistant, response));
    Console.WriteLine();
}
```




Local Machine
Ollama Model Catalog
ollama pull <model>
ollama run <model>
Ollama Management Service
`ollama serve`
ollama list / ollama ls
ollama ps (check process)
ollama rm <model>
Client / App
Hugging Face (only GGUF format)
ollama pull <model.gguf>
Localhost / inferencing
http://localhost:11434/v1
CPU
GPU (Initial Setting)

Wanna run Ollama with CPU? Need to run with Docker

```
(base) PS C:\Users\boatc> docker exec -it ollama ollama run qwen2.5:1.5b
>>> Who is Lewis Hamilton?
Lewis Hamilton is an English former professional racing driver who is currently the Formula One World Champion and
holds multiple other honors for his achievements in motorsports.

Here's some key information about him:

1. He was born on July 3, 1985, in Stoke-on-Trent, England.

2. Hamilton began driving a kart when he was just five years old and went on to become the youngest driver at the
Formula One World Championship.

3. He has won seven Grand Prix titles across different championships (Formula One, World Drivers' Championships,
and World Constructors' Championships).

4. In 2015, he became only the second British driver in history to win a World Drivers' Championship after Jim
Clark's victory with Lotus during the 1963 season.

5. Hamilton has been named as one of Time magazine's "100 Most Influential People in the World" four times and was
voted into Forbes' list of the world's richest people for three consecutive years (2015-2017).

6. As a motivational speaker, he has spoken at events like the United Nations General Assembly and given inspiring
talks to audiences worldwide.

Hamilton is known not only for his racing achievements but also as an advocate for various causes such as climate
change awareness and supporting women in motorsports through initiatives like "Hamilton's Garage."

It's worth noting that there are multiple drivers named Lewis Hamilton, which sometimes leads to confusion. The
two main drivers should be distinguished based on the context or specific event being discussed.

>>>
(base) PS C:\Users\boatc> docker exec -it ollama ollama ps
NAME            ID              SIZE      PROCESSOR    UNTIL
qwen2.5:1.5b    65ec06548149    1.6 GB    100% CPU     4 minutes from now
(base) PS C:\Users\boatc>
```

Or setting in MODELFILE of the model

```
3. He has won seven Grand Prix titles across different championships (Formula
and World Constructors' Championships).

4. In 2015, he became only the second British driver in history to win a Worl
Clark's victory with Lotus during the 1963 season.

5. Hamilton has been named as one of Time magazine's "100 Most Influential Pe
voted into Forbes' list of the world's richest people for three consecutive y

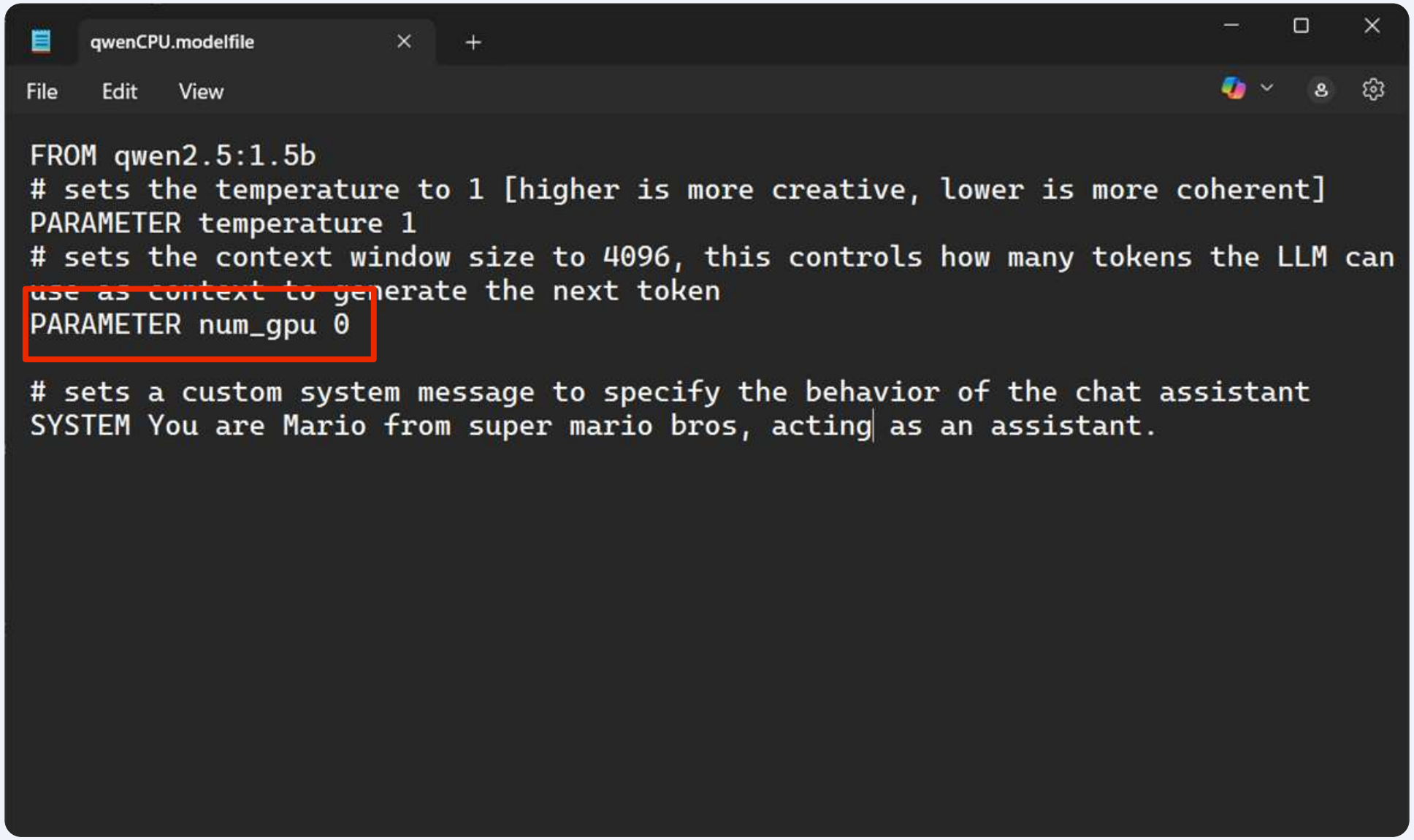
6. As a motivational speaker, he has spoken at events like the United Nations
talks to audiences worldwide.

Hamilton is known not only for his racing achievements but also as an advocat
change awareness and supporting women in motorsports through initiatives like

It's worth noting that there are multiple drivers named Lewis Hamilton, which
two main drivers should be distinguished based on the context or specific eve

>>>
(base) PS C:\Users\boatc> docker exec -it ollama ollama ps
NAME              ID              SIZE      PROCESSOR    UNTIL
qwen2.5:1.5b      65ec06548149    1.6 GB    100% CPU     4 minutes from now
(base) PS C:\Users\boatc>
```

Or setting in MODELFILE of the model



We are bringing our assets together to democratize AI



Foundry with cloud hosted models



ONNX Runtime (ORT): cross-platform on-device inferencing



Windows on-device model APIs & ML runtime

70K+
Organizations

4x
Acceleration

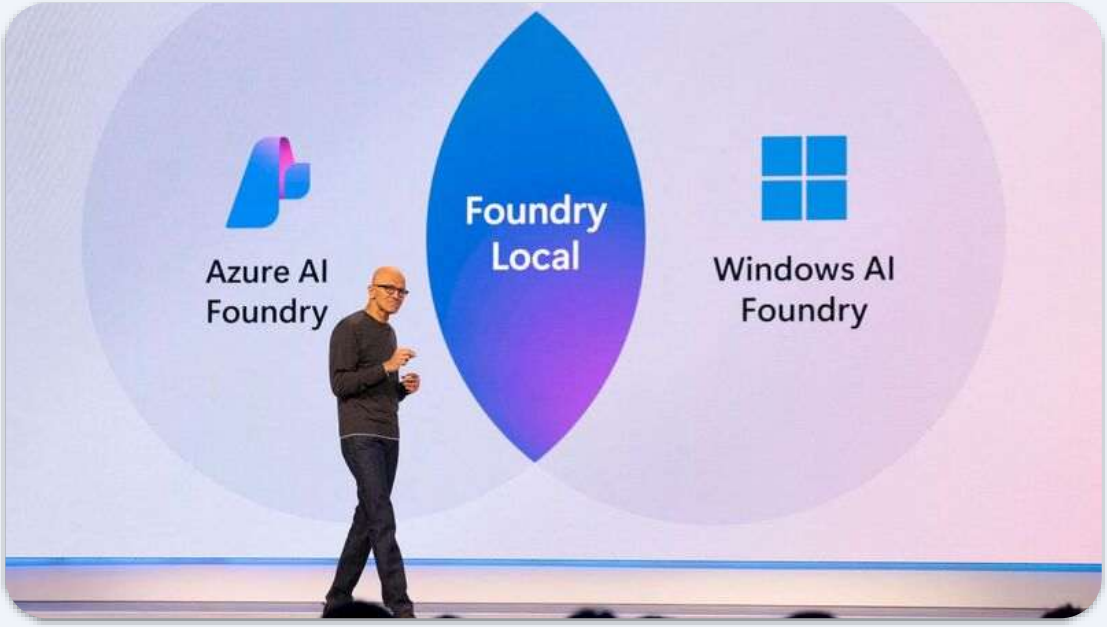
100s of millions
Devices

In-Preview

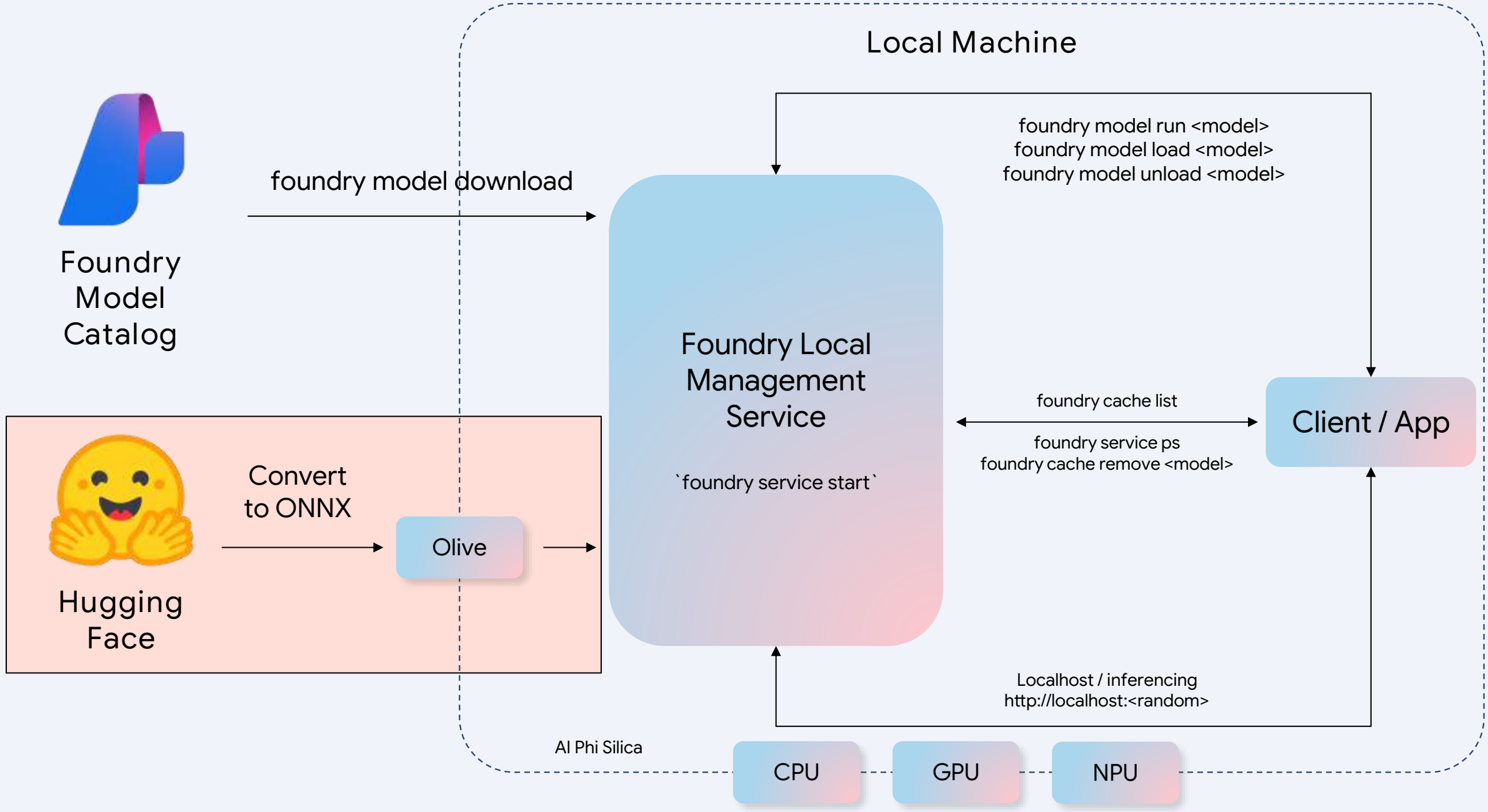
Microsoft Foundry Local

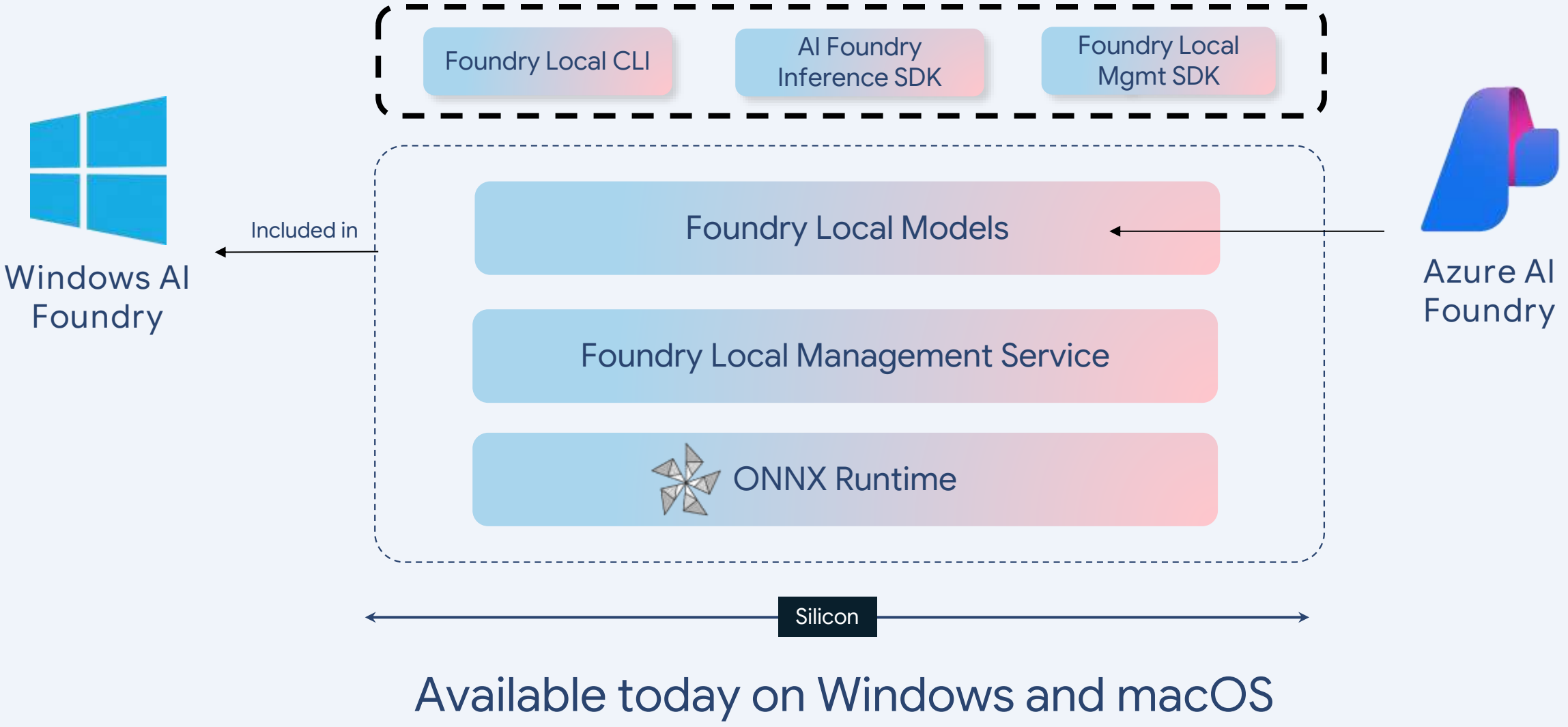
Ship cross-platform performant apps with enterprise-grade local AI effortlessly

> **Important**
>
> - Foundry Local is available in preview. Public preview releases provide early access to features that are in active deployment.
> - Features, approaches, and processes can change or have limited capabilities, before General Availability (GA).



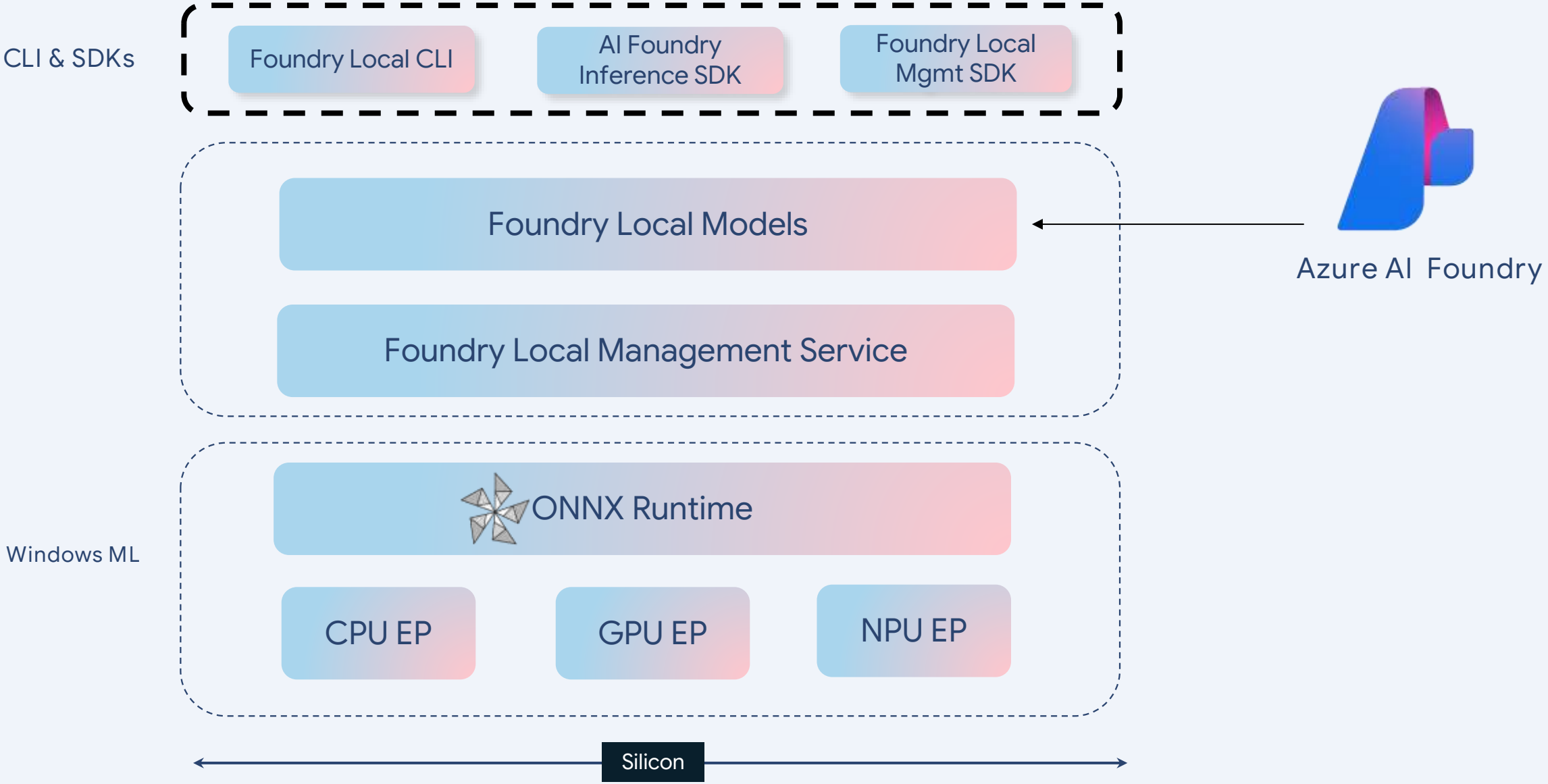
Microsoft CEO Satya Nadella (Photo by JASON REDMOND/AFP via Getty Images)


Local Machine
Foundry Model Catalog
foundry model download
foundry model run <model>
foundry model load <model>
foundry model unload <model>
Foundry Local Management Service
`foundry service start`
foundry cache list
foundry service ps
foundry cache remove <model>
Client / App
Hugging Face
Convert to ONNX
Olive
Localhost / inferencing
http://localhost:<random>
AI Phi Silica
CPU
GPU
NPU



Available today on Windows and macOS

Foundry Local is integrated in Windows AI Foundry



Foundry Local Management SDK

A lightweight, cross-platform toolkit for building high-performance local AI apps

- Python, JavaScript, .NET (Newest Released)
- Foundry Local Service Management
- Model Management
- Model Cache Management
- Catalog Management

- Get Started
  - Python: pip install foundry-manager-sdk
  - JS/TS: npm install foundry-local-sdk
  - .NET: put the following XML file to the project

```
<Project Sdk="Microsoft.NET.Sdk">


  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0-
windows10.0.26100</TargetFramework>
    <RootNamespace>hello-foundry-
local</RootNamespace>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <WindowsAppSDKSelfContained>true</WindowsAp
pSDKSelfContained>
  </PropertyGroup>


  <ItemGroup>
    <PackageReference
Include="Microsoft.AI.Foundry.Local.WinML"
Version="0.8.0" />
    <PackageReference
Include="Microsoft.Extensions.Logging"
Version="9.0.10" />
    <PackageReference Include="OpenAI"
Version="2.5.0" />
  </ItemGroup>


</Project>
```

```
using Microsoft.AI.Foundry.Local;
using Microsoft.Extensions.Logging;
using OpenAI;
using System.ClientModel;

var config = new Configuration
{
    AppName = "my-app-name",
    LogLevel = Microsoft.AI.Foundry.Local.LogLevel.Information,
    Web = new Configuration.WebService
    {
        Urls = "http://127.0.0.1:55588"
    }
};

using var loggerFactory = LoggerFactory.Create(builder =>
{
    builder.SetMinimumLevel(Microsoft.Extensions.Logging.LogLevel.Information);
});

var logger = loggerFactory.CreateLogger<Program>();

// Initialize the singleton instance.
await FoundryLocalManager.CreateAsync(config, logger);
var mgr = FoundryLocalManager.Instance;

// Get the model catalog
var catalog = await mgr.GetCatalogAsync();

// Get a model using an alias
var model = await catalog.GetModelAsync("qwen2.5-0.5b") ?? throw new Exception("Model not found");
```

```
// Download the model (the method skips download if already cached)
await model.DownloadAsync(progress ⇒
{
    Console.Write($"\rDownloading model: {progress:F2}%");
    if (progress ⩾ 100f)
    {
        Console.WriteLine();
    }
});


// Load the model
await model.LoadAsync();


// Start the web service
await mgr.StartWebServiceAsync();
```

```
// <<<<<< OPEN AI SDK USAGE >>>>>>
// Use the OpenAI SDK to call the local Foundry web service


ApiKeyCredential key = new ApiKeyCredential("notneeded");
OpenAIClient client = new OpenAIClient(key, new OpenAIClientOptions
{
    Endpoint = new Uri(config.Web.Urls + "/v1"),
});


var chatClient = client.GetChatClient(model.Id);


var completionUpdates = chatClient.CompleteChatStreaming("Why is the sky blue?");


Console.Write($"[ASSISTANT]: ");
foreach (var completionUpdate in completionUpdates)
{
    if (completionUpdate.ContentUpdate.Count > 0)
    {
        Console.Write(completionUpdate.ContentUpdate[0].Text);
    }
}
Console.WriteLine();
// <<<<<< END OPEN AI SDK USAGE >>>>>>

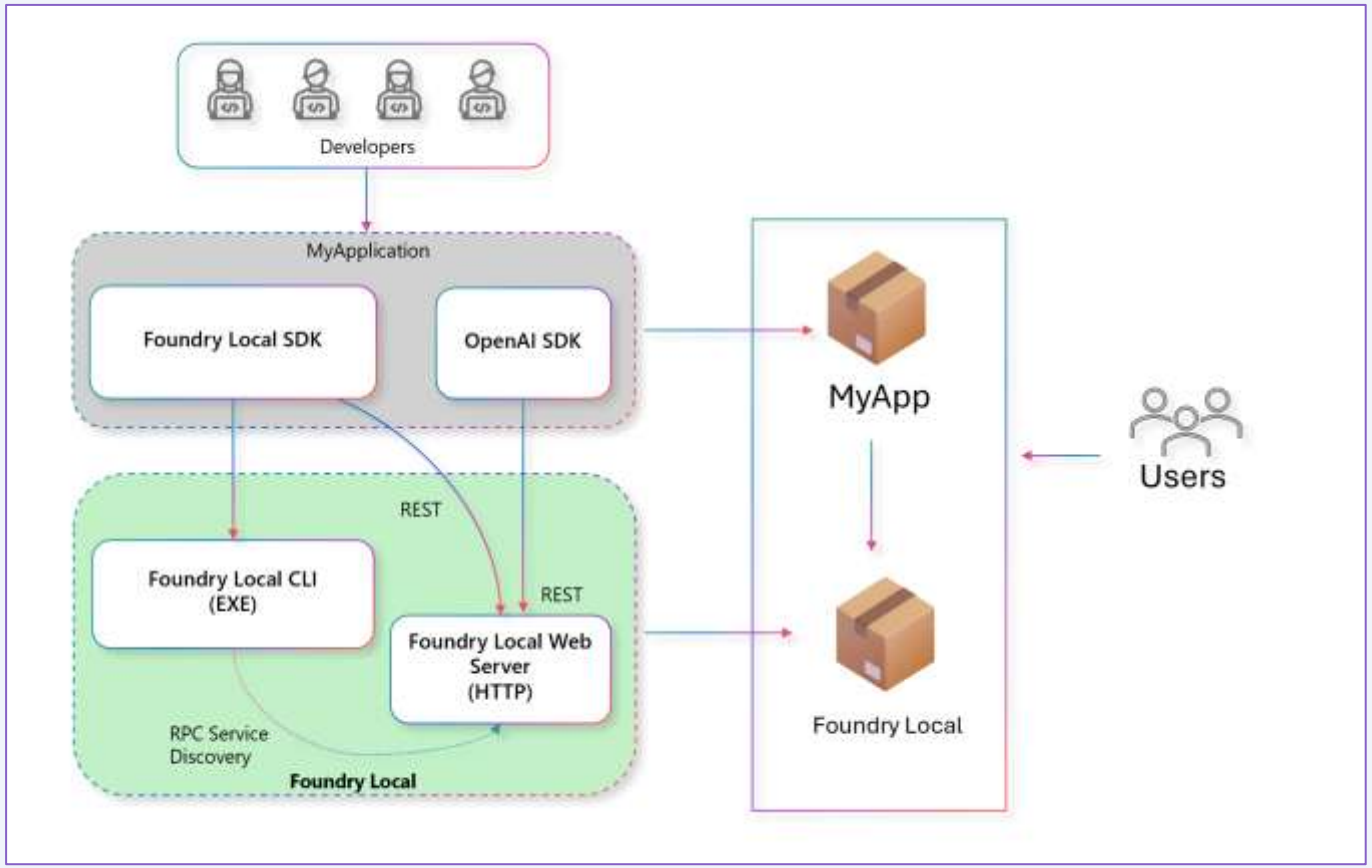

// Tidy up
// Stop the web service and unload model
await mgr.StopWebServiceAsync();
await model.UnloadAsync();
```

Comparing the Architecture

Old Architecture

The SDK would use a Remote Procedural Call (RPC) to find Foundry Local CLI executable on the machine, start the webserver, and then communicate with it over HTTP. This architecture had several limitations, including:

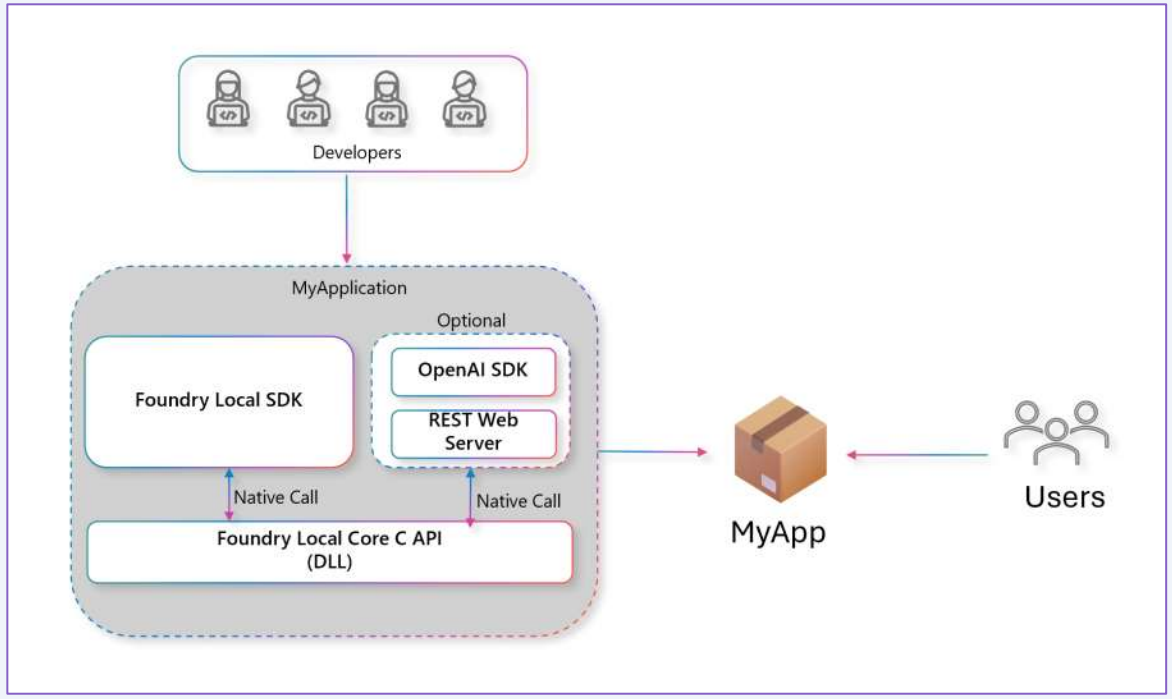
- Complexity in managing the webserver lifecycle.
- Challenging deployment: End users needed to have the Foundry Local CLI installed on their machines *and* your application.
- Version management of the CLI and SDK could lead to compatibility issues.



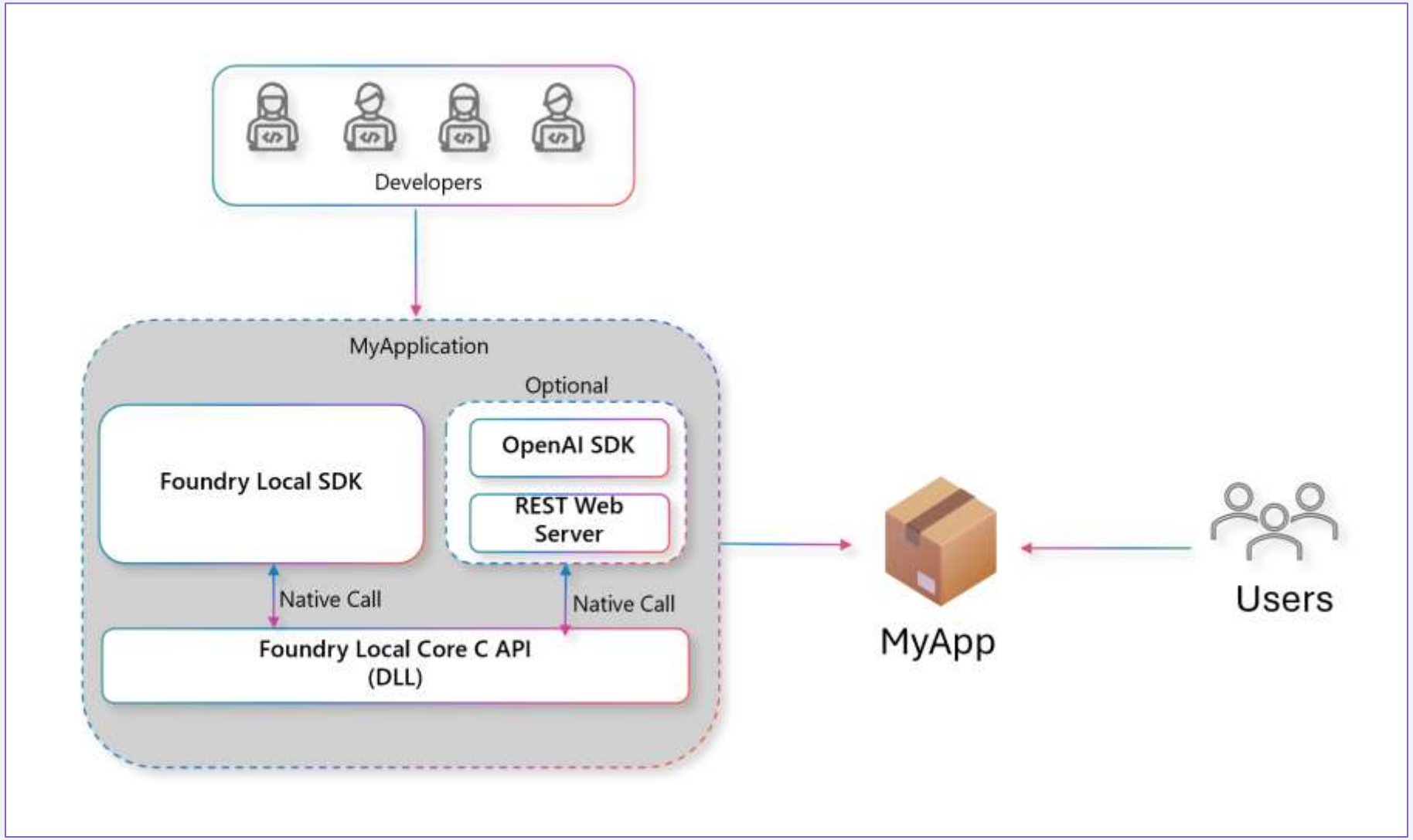
Comparing the Architecture

New Architecture

- Your application is self-contained. It doesn't require the Foundry Local CLI to be installed separately on the end user's machine making it easier for you to deploy applications.
- The REST web server is *optional*. You can still use the web server if you want to integrate with other tools that communicate over HTTP. Read Use chat completions via REST server with Foundry Local for details on how to use this feature.
- The SDK has native support for chat completions and audio transcriptions, allowing you to build conversational AI applications with fewer dependencies. Read Use Foundry Local native chat completions API for details on how to use this feature.
- On Windows devices, you can use a Windows ML build that handles hardware acceleration for models on the device by pulling in the right runtime and drivers.



Comparing the Architecture



.NET Tools & Techniques for Sustainable SLMs

Measurement and Efficient Deployment For Hardware Consumption

a. Measurement and Monitoring
    a. "System.Diagnostics.PerformanceCounter" Version="8.0.0"
    b. "Microsoft.Extensions.Diagnostics.ResourceMonitoring" Version="8.0.0" />

b. Deployment / Inference
    i. deploy as ONNX Format (For on-edge Usage)
    ii. deploy as GGUF Format (For Enterprise Usage)

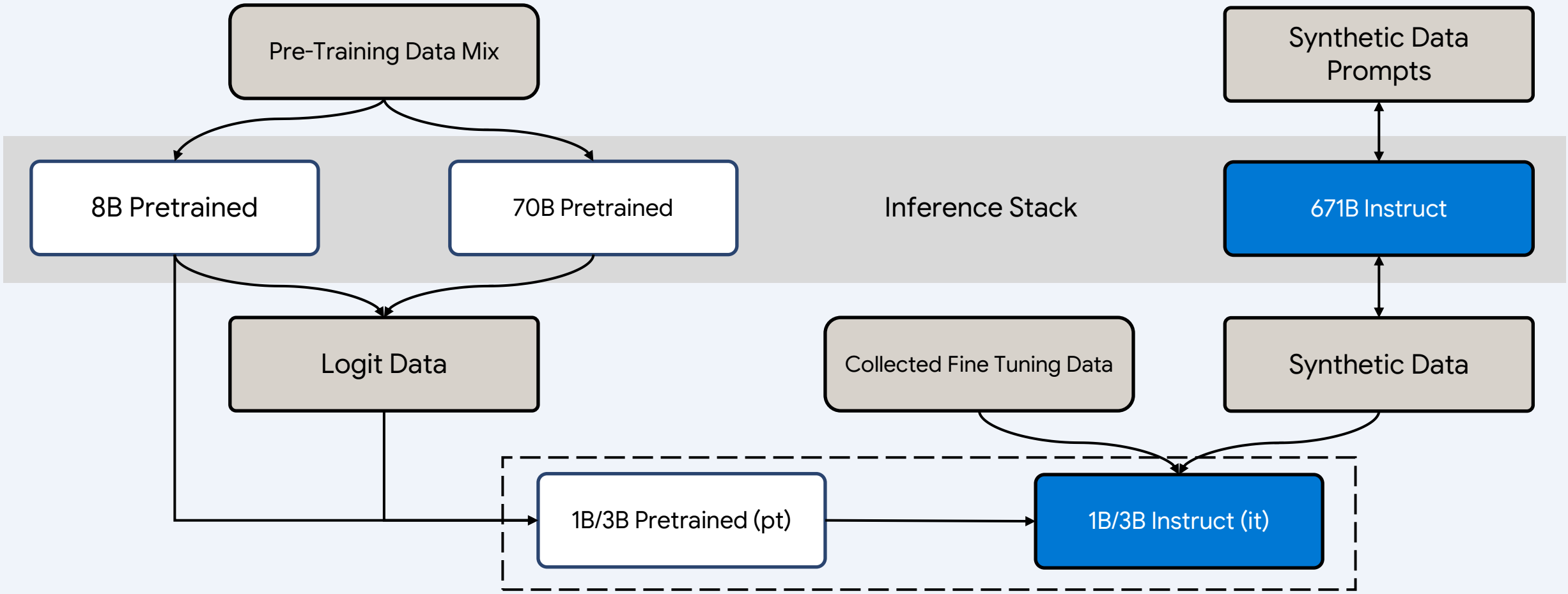
Inference Optimization Techniques

- Reduce Compute
  - Techniques like caching, knowledge distillation, and early exit, each of them employing distinct strategies to reduce computation. (Part of Python)

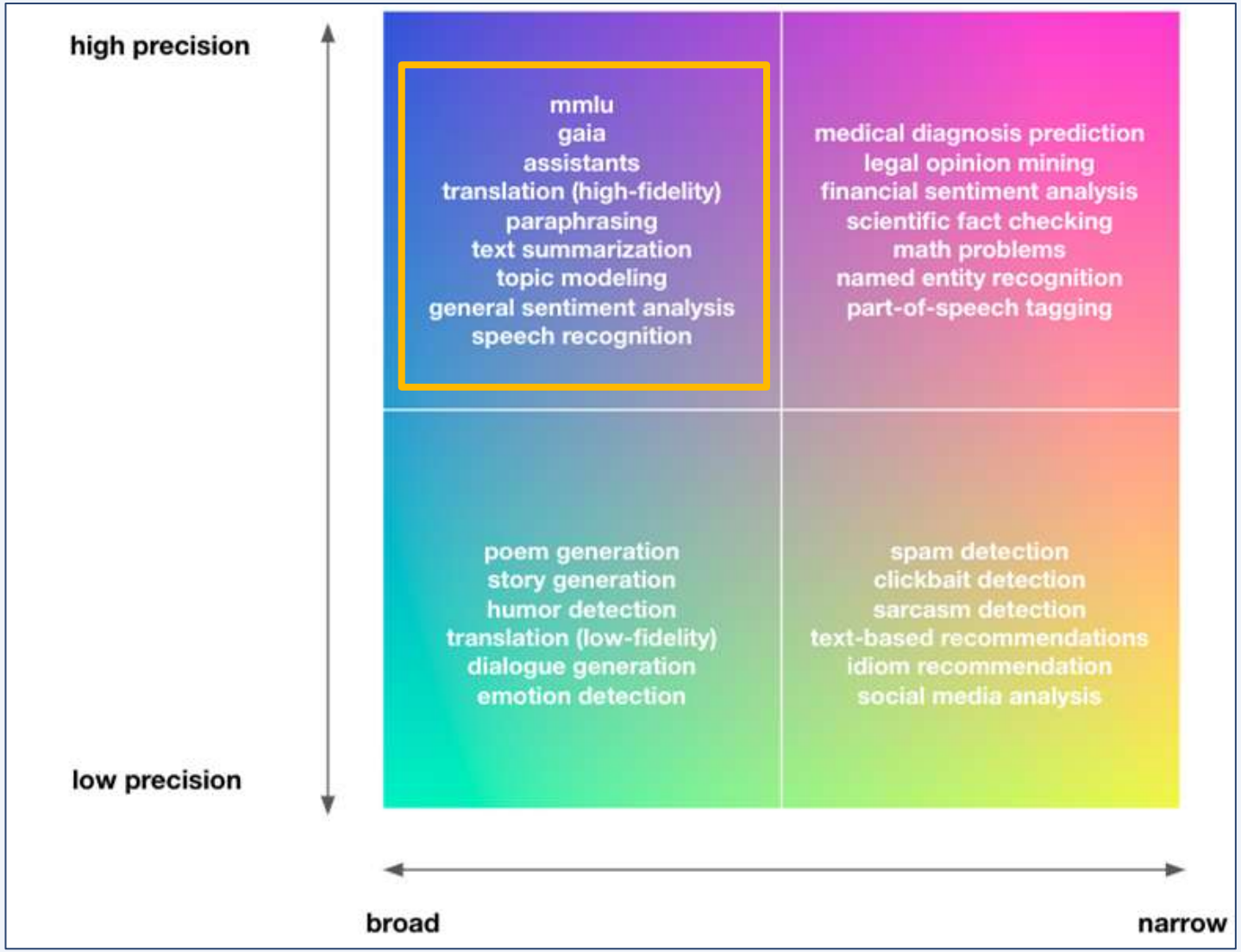
- Speed-up Decoding
  - Techniques for parallel and speculative decoding aim to improve the throughput of the model: the number of tokens generated per second.
    (i.e. Microsoft.ML.Tokenizers)

- Reduce storage needs
  - Quantization techniques aim to reduce the amount of storage needed for weights and activations of the model, by reducing space required to store numbers from 32 bits to 16, 8, or even 4 bits. (ORT – ONNX Runtime will support on this part)

Key Concepts - Distillation From Meta's Llama



Early-days Limitation



Quantization

Quantization reduces the number of bits required to represent the parameters of an LLM while attempting to maintain most of the original information. This comes with some loss in precision but often makes up for it as the model is much faster to run, requires less VRAM, and is often almost as accurate as the original.

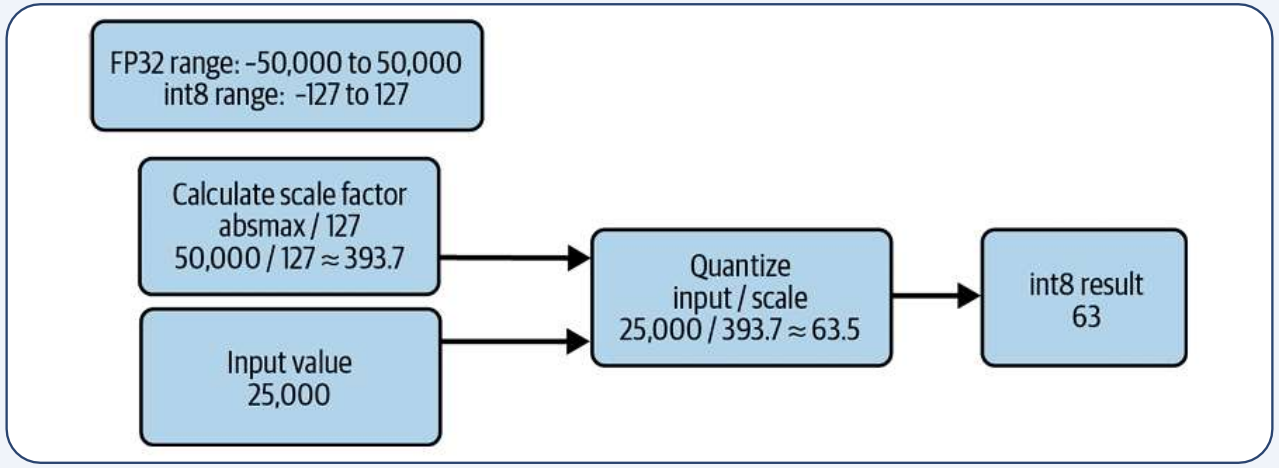


Fig. Absmax quantization

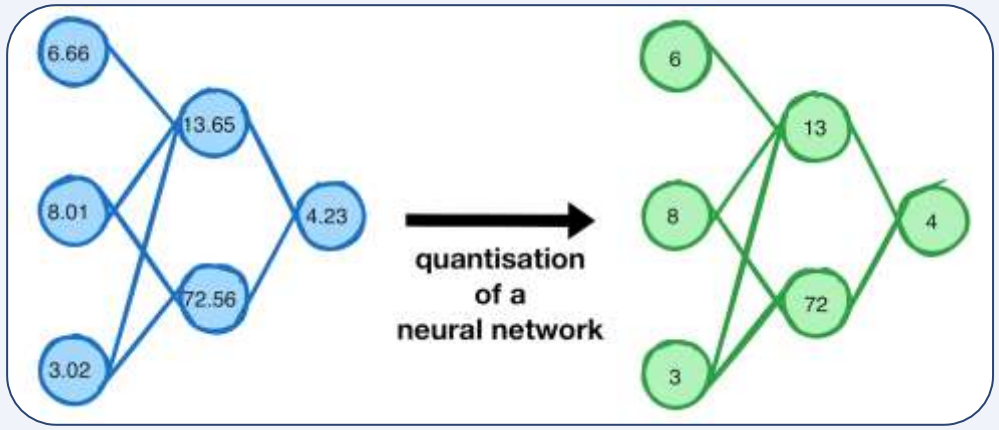


Fig. Quantization of a neural network

ℹ️ Take note that ...

- Symmetric quantization always maps zero to zero, while asymmetric quantization allows for a non-zero offset (zero-point) to be included in the mapping.

Read more at
https://www.maartengrootendorst.com/blog/quantization/

Acceleration Across Silica or Optimization For Limited Resources

Windows

macOS



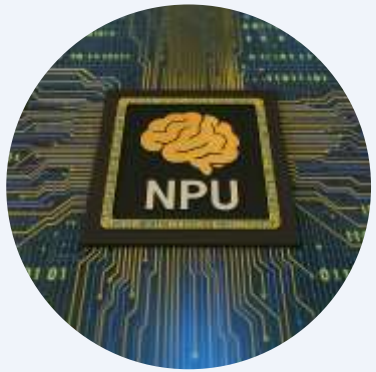
CPU



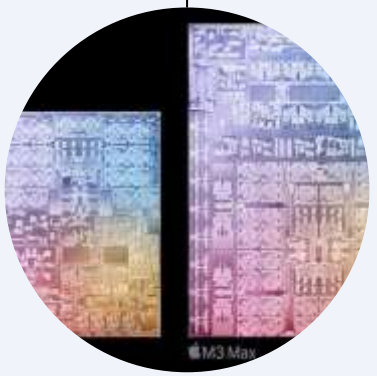
Integrated GPU



Dedicated GPU

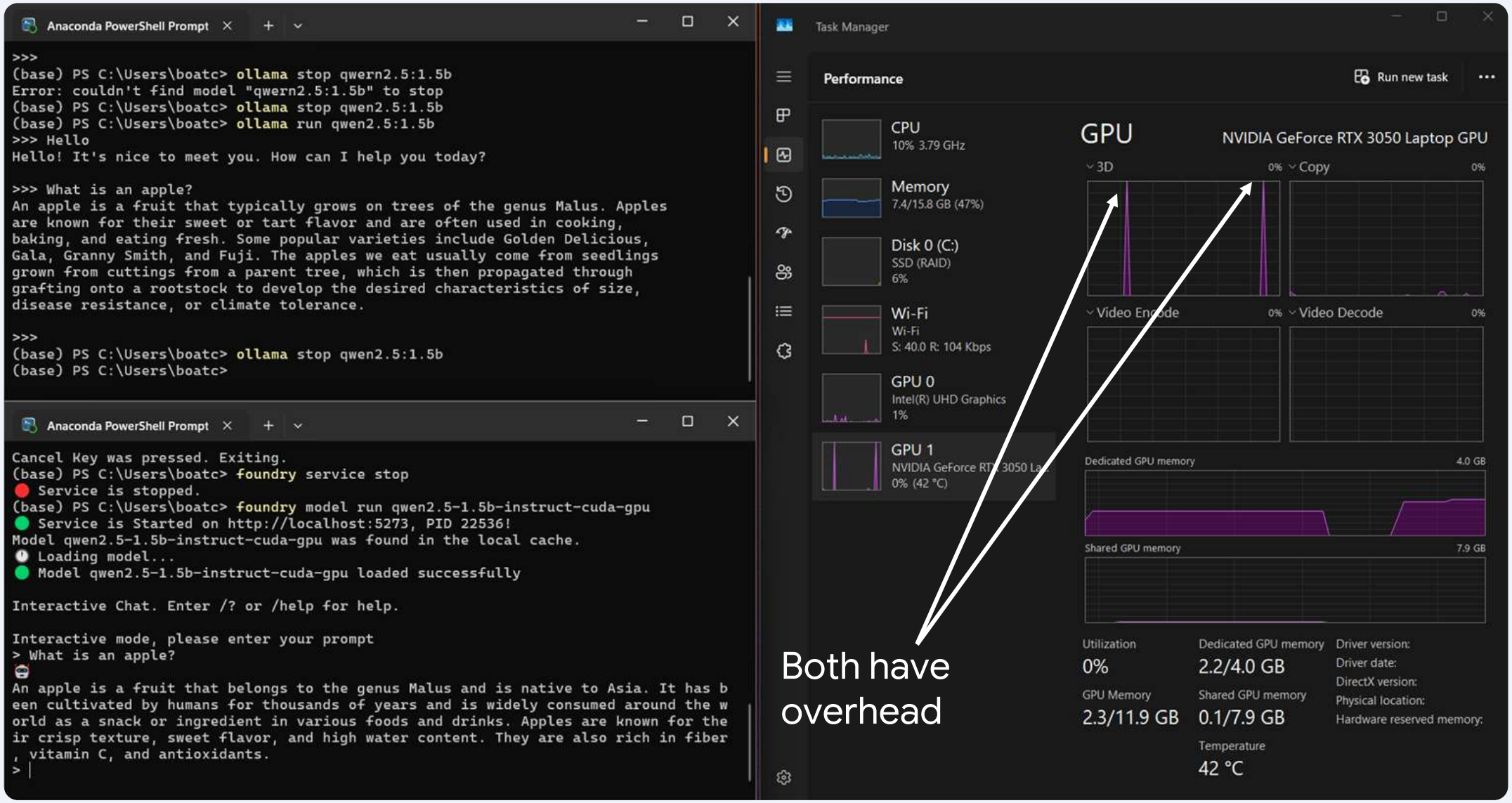


NPU

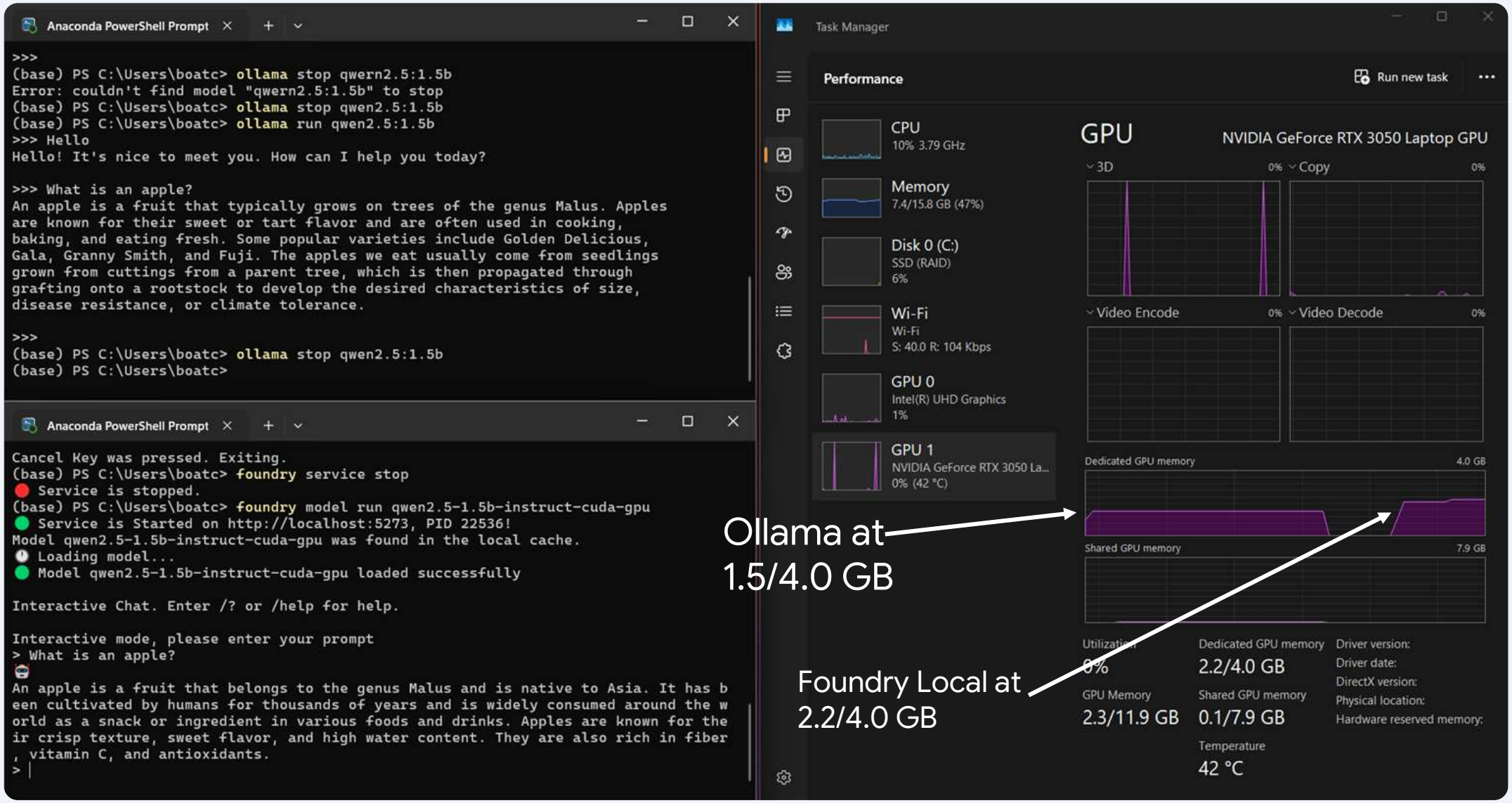


Apple Silicon

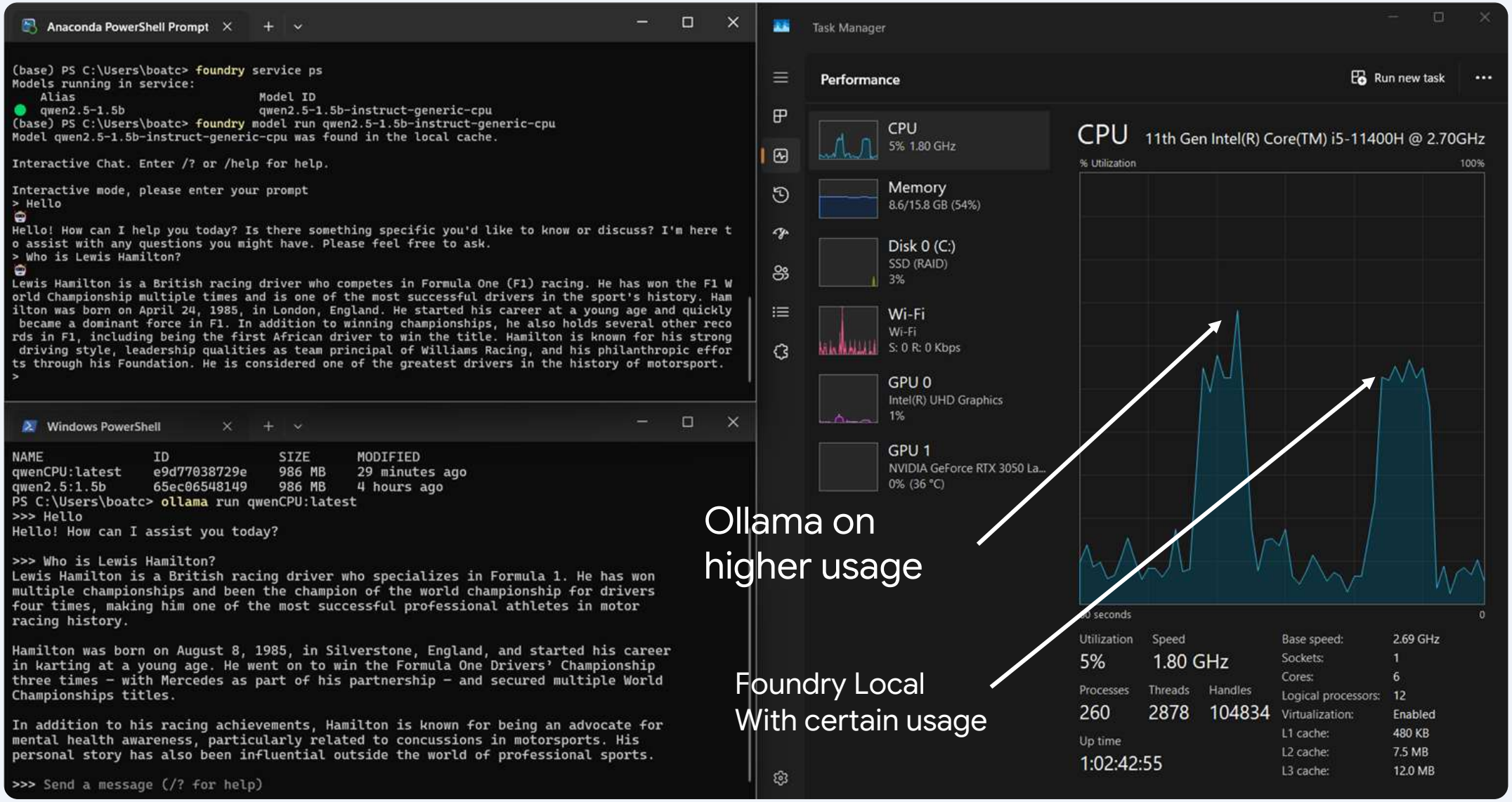
GPU Comparison



GPU Comparison



CPU Comparison



Let’s do it in .NET Way

From Microsoft Official Packages

System.Diagnostics.PerformanceCounter

This package provides types that allow applications to interact with the Windows performance counters.

- Windows allows you to examine how programs you run affect your computer's performance, both in real time and by collecting log data for later analysis. You can do this via the Windows Performance Monitor tool, which uses performance counters, among other features.

- Windows performance counters provide a high-level abstraction layer that provides a consistent interface for collecting various kinds of system data such as CPU, memory, and disk usage. They can be included in the operating system or can be part of individual applications. Windows Performance Monitor requests the current value of performance counters at specified time intervals.

- System administrators often use performance counters to monitor systems for performance or behavior problems. Software developers often use performance counters to examine the resource usage of their programs.

Visit https://www.nuget.org/packages/system.diagnostics.performancecounter/ for more information

System.Diagnostics.PerformanceCounter

Key Features

- Can be used to read existing predefined or custom counters.
- Can be used for publishing (writing) data to custom counters.
- Can collect performance counters from the local machine or from a remote machine.

Visit https://www.nuget.org/packages/system.diagnostics.performancecounter/ for more information

Resource monitoring

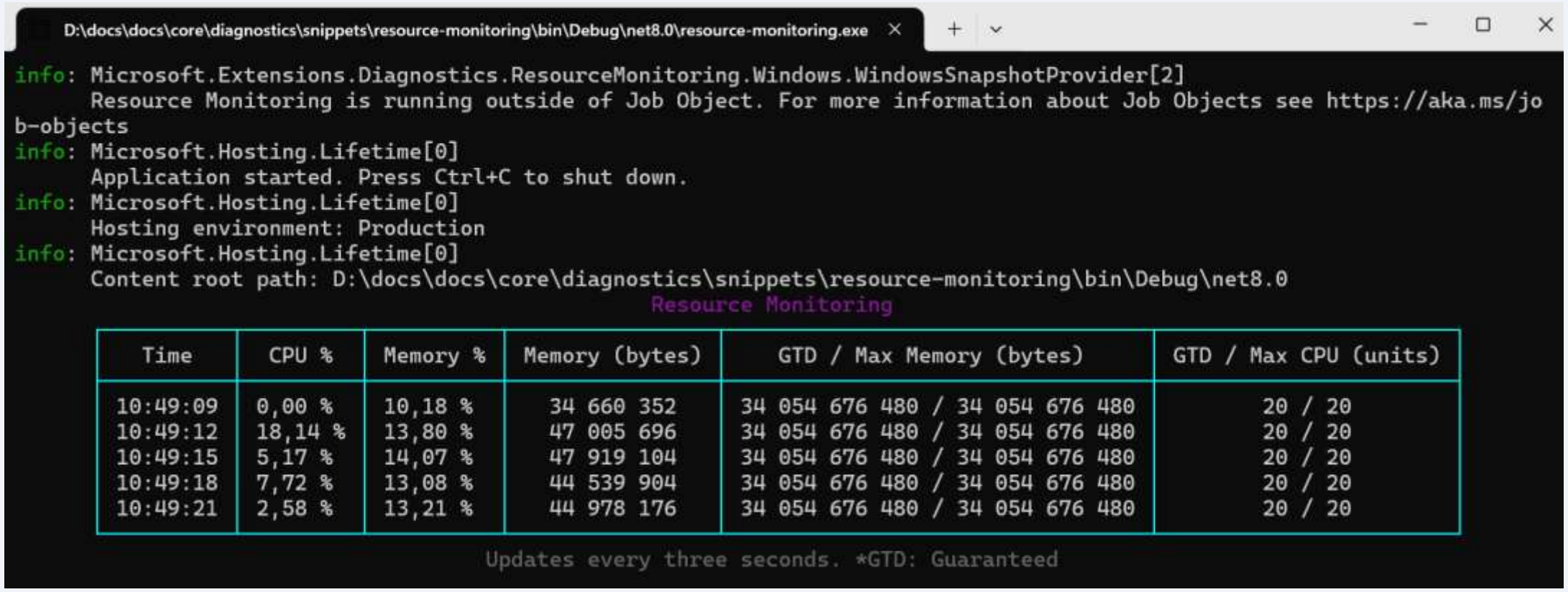
Nickname of Microsoft.Extensions.Diagnostics.ResourceMonitoring

- Resource monitoring involves the continuous measurement of resource utilization values, such as CPU, memory, and network usage. The Microsoft.Extensions.Diagnostics.ResourceMonitoring NuGet package offers a collection of APIs tailored for monitoring the resource utilization of your .NET applications.

- The measurements can be consumed in two ways:
  - Using .NET metrics.
  - Using the IResourceMonitor interface. This interface is deprecated, so use the metrics-based approach instead. If you still need to listen to metric values manually, see Migrate to metrics-based resource monitoring.

https://learn.microsoft.com/en-us/dotnet/core/diagnostics/diagnostic-resource-monitoring

Resource monitoring



Use the IResourceMonitor interface

Demo Section

Experimenting The Hardware Consumption (CPU Usage, Memory Peak) on Small Language Model (Qwen 2.5:1.5b) between Ollama (GGUF Format) & Foundry Local (ONNX Format)

To Conclude...

While GPUs generally offer a significant speed and efficiency advantage for running LLMs due to their parallel processing capabilities, in specific scenarios, CPUs can sometimes outperform GPUs, particularly for on-device AI and smaller models. This is because factors like GPU memory transfer overhead and CPU thread optimization can play a crucial role, especially in mobile devices.

In mobile devices or other limited resource environments, CPUs can be a viable and sometimes even superior option for running LLMs due to their ability to leverage optimized thread usage and potentially avoid GPU overhead.

The Road Ahead:
for Developing AI Applications

What is it good for ONNX Format?

Along with using ONNX Runtime for Device Optimization

Use ONNX Runtime for high performance, scalability, and flexibility when deploying generative AI models. With support for diverse frameworks and hardware acceleration, ONNX Runtime ensures efficient, cost-effective model inference across platforms.

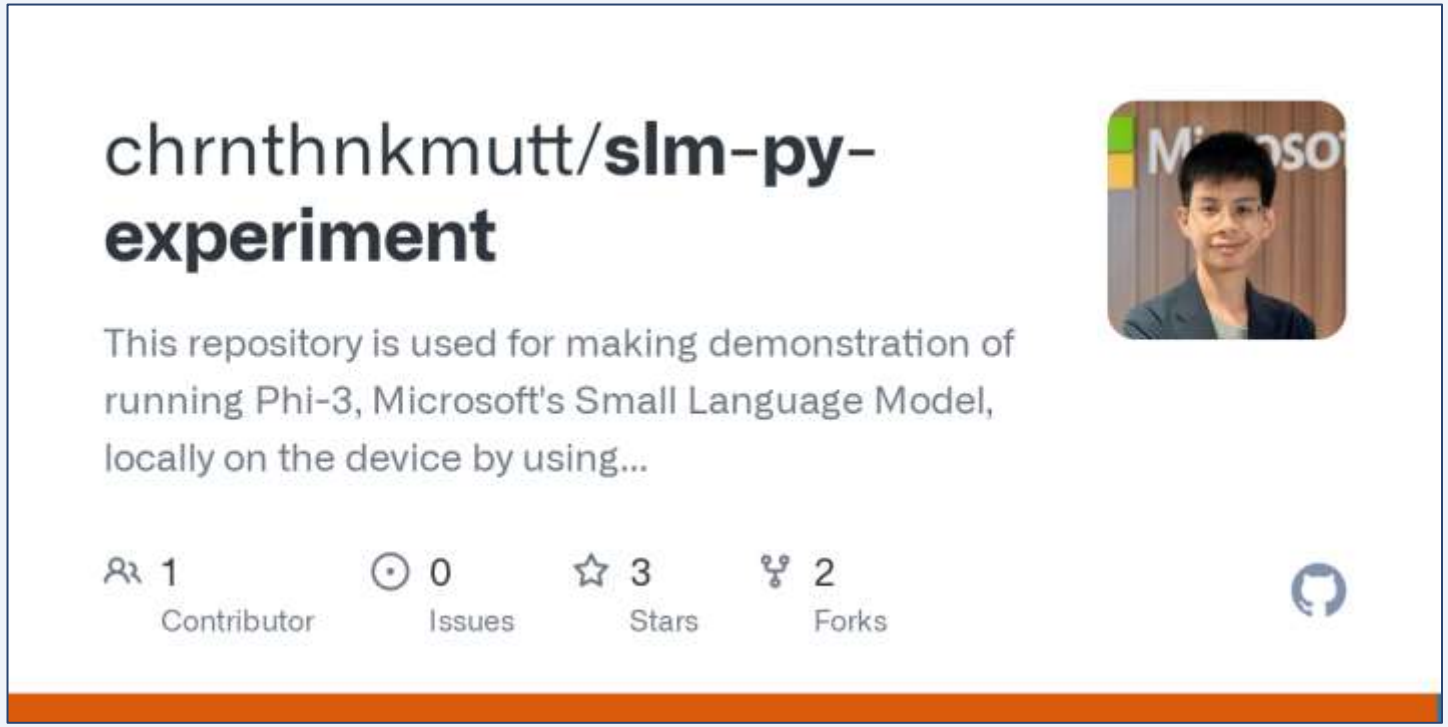






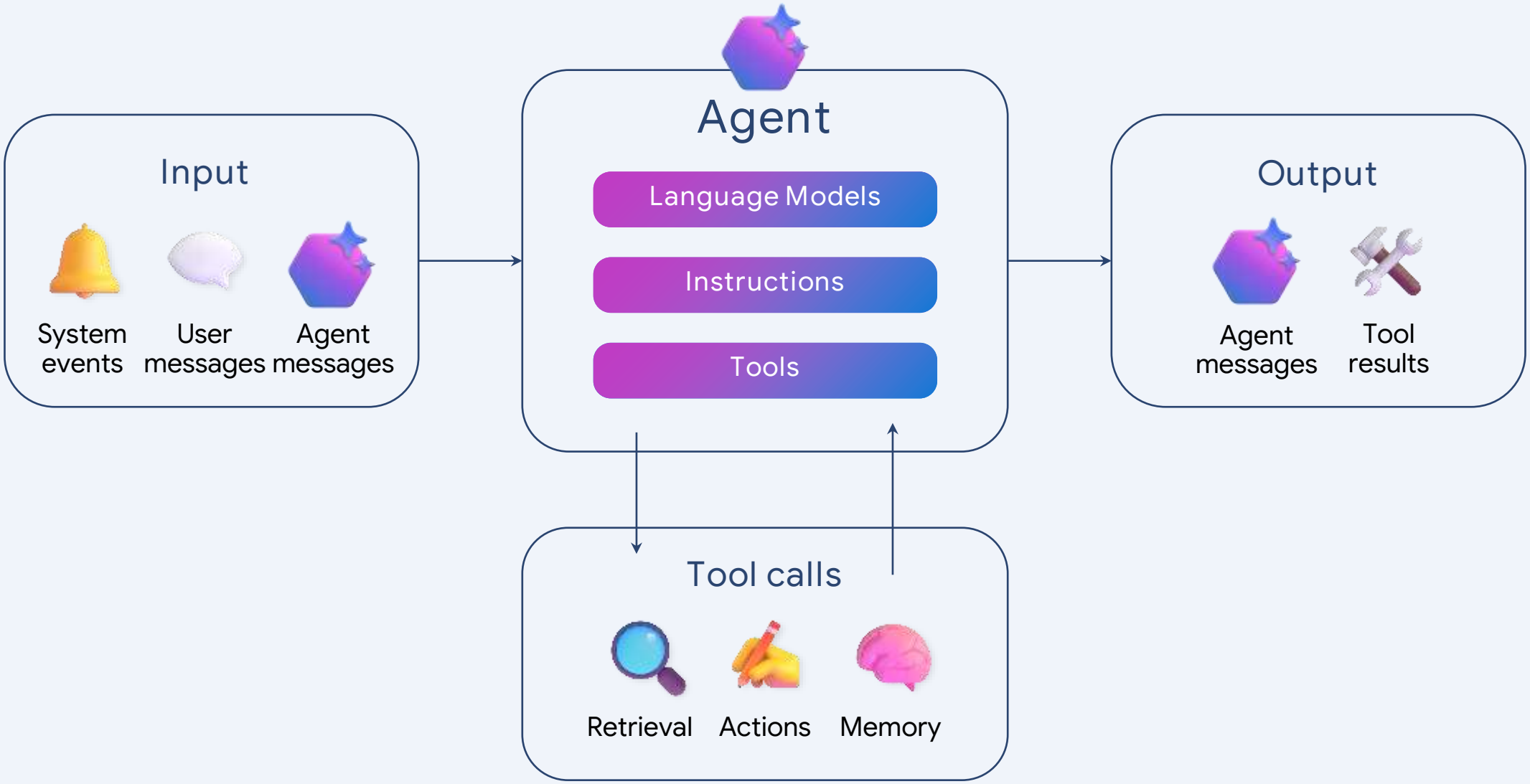
Make the experiment by yourself...

By cloning this repository to test on your laptop or PC.

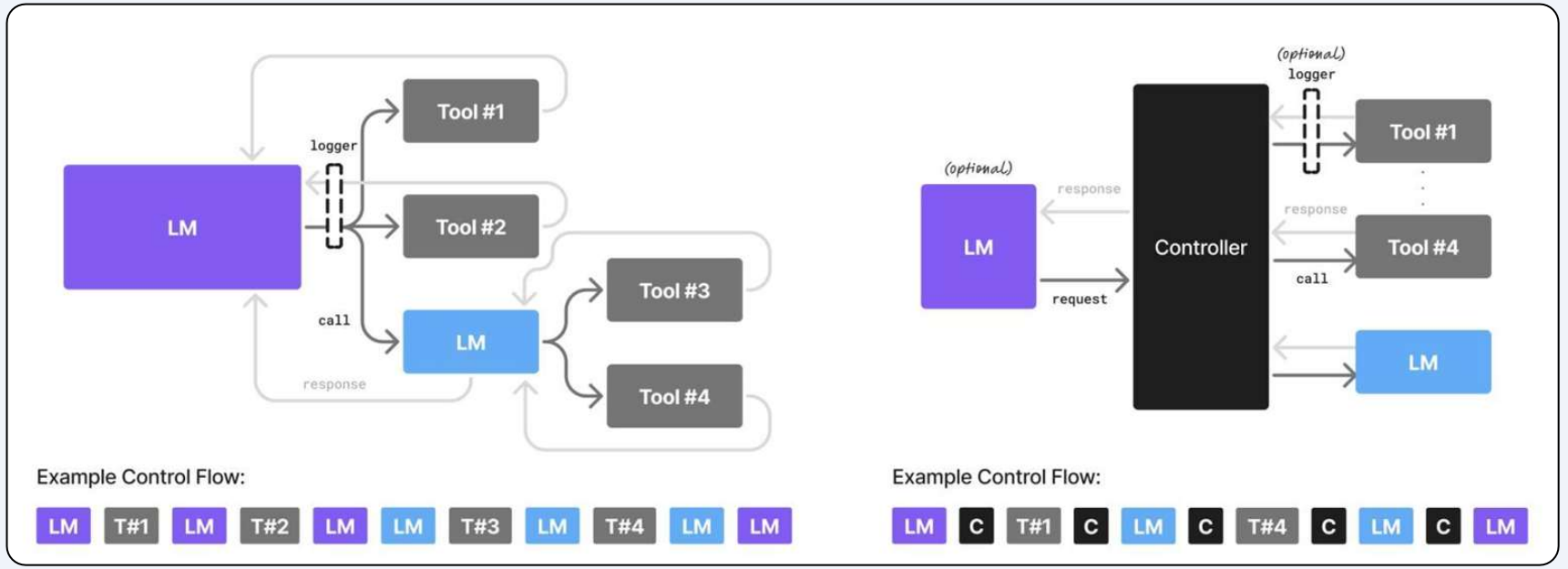


https://github.com/chrnthnkmutt/slm_dotnet_experiment/tree/main/dotnet_slm_diagnostic

Overview of Agent System



Proposed Architecture from NVIDIA



An illustration of agentic systems with different modes of agency. Left: Language model agency. The language model acts both as the HCI and the orchestrator of tool calls to carry out a task. Right: Code agency. The language model fills the role of the HCI (optionally) while a dedicated controller code orchestrates all interactions.

Small Language Models are the Future of Agentic AI
Peter Belcak, Greg Heinrich, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, Pavlo Molchanov
NVIDIA Research

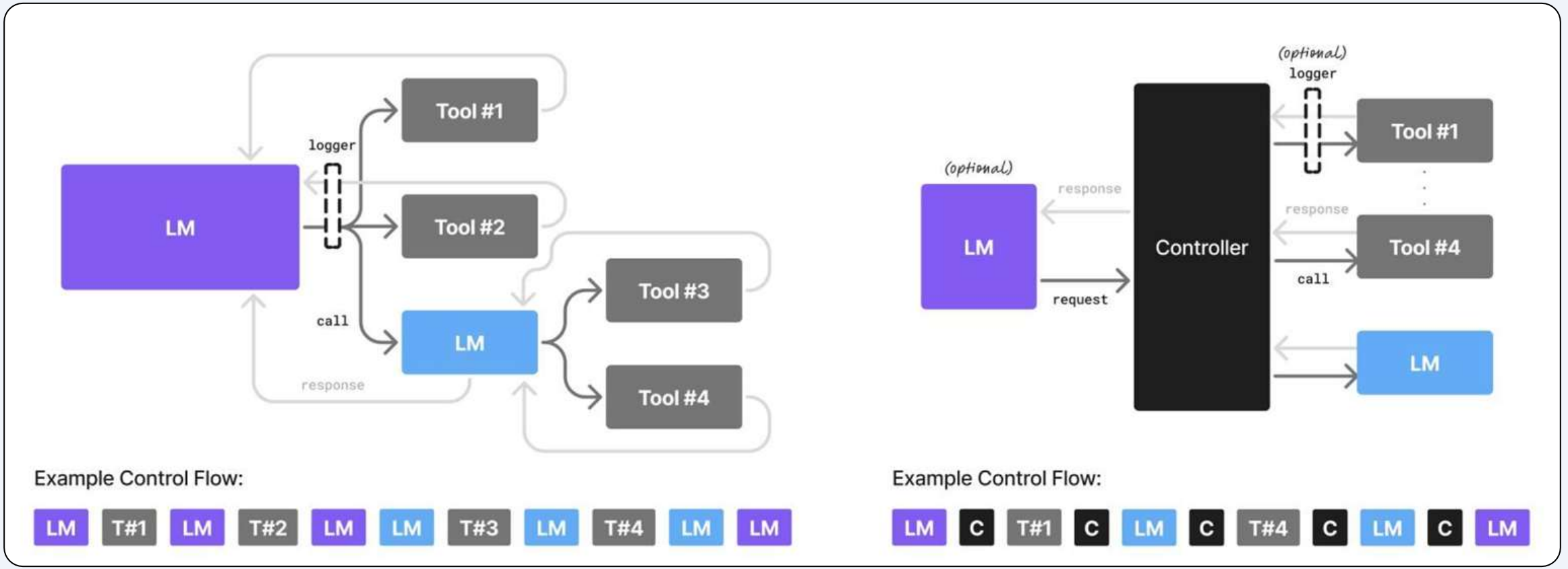


Small (or Large) Language Models for Agentic AI (LM agency vs. Code agency)

Deploying AI agents at an economically scalable level—instead of using an LM (Language Model) as the direct controller of the agent (where it has to interpret every situation), we use rule-based code for control, and the (L/S) LM acts as an interpreter instead. This interpreter can either be an online LLM or an offline SLM.

- Asst. Prof. Dr. Santitham Prom-on, Head of Computer Engineering Department, CPE KMUTT

Proposed Architecture from NVIDIA



An illustration of agentic systems with different modes of agency. Left: Language model agency. The language model acts both as the HCI and the orchestrator of tool calls to carry out a task. Right: Code agency. The language model fills the role of the HCI (optionally) while a dedicated controller code orchestrates all interactions.

Small Language Models are the Future of Agentic AI

Peter Belcak, Greg Heinrich, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, Pavlo Molchanov - NVIDIA Research

Microsoft

Thank You For Watching!
& To-be Continued at
Microsoft Ignite After Party
For More Updates

Feel free to contact backstage at any platforms!



Charunthon Limseelo (@boatchrnthn)